

FastAPI Interview Questions & Answers (50)

Q1. What is FastAPI?

Answer: A modern Python web framework for building high-performance APIs with automatic validation and documentation.

Q2. Why is FastAPI fast?

Answer: It is built on ASGI, Starlette, and Pydantic, and supports asynchronous programming.

Q3. Main features?

Answer: Type hints, validation, async support, OpenAPI docs, dependency injection.

Q4. What is ASGI?

Answer: The asynchronous server interface used by FastAPI.

Q5. FastAPI vs Flask?

Answer: FastAPI has built-in validation, async support, and docs; Flask is simpler but needs extensions.

Q6. FastAPI vs Django?

Answer: Django is a full-stack framework; FastAPI focuses on APIs and performance.

Q7. What is Uvicorn?

Answer: An ASGI server used to run FastAPI applications.

Q8. What is Pydantic?

Answer: A library for data validation using Python type hints.

Q9. Request model?

Answer: A Pydantic model that validates incoming request data.

Q10. Response model?

Answer: A Pydantic model defining the structure of API responses.

Q11. Path parameters?

Answer: Values embedded in the URL, e.g. /users/{id}.

Q12. Query parameters?

Answer: Values after ?, e.g. /users?limit=10.

Q13. PUT vs PATCH?

Answer: PUT replaces an entire resource; PATCH updates part of it.

Q14. POST vs PUT?

Answer: POST creates a resource; PUT updates or replaces one.

Q15. HTTP status codes?

Answer: Numeric codes indicating request results, e.g. 200, 201, 404, 500.

Q16. Dependency injection?

Answer: A way to provide shared objects like database sessions automatically.

Q17. Depends()?

Answer: Requests a dependency such as a database session.

Q18. Why get_db()?

Answer: To create and close a database session safely per request.

Q19. Why yield?

Answer: It allows cleanup code like db.close() to run after the request.

Q20. SQLAlchemy?

Answer: A Python ORM for interacting with relational databases.

Q21. ORM?

Answer: Maps Python classes to database tables.

Q22. SessionLocal?

Answer: Factory that creates database sessions.

Q23. Engine?

Answer: Connects SQLAlchemy to the database.

Q24. Base?

Answer: Base class used to define ORM models.

Q25. create_all()?

Answer: Creates tables that do not already exist.

Q26. Models vs Schemas?

Answer: Models map to database tables; schemas validate API data.

Q27. CRUD?

Answer: Create, Read, Update, Delete operations.

Q28. db.add()?

Answer: Marks a new object for insertion.

Q29. db.commit()?

Answer: Permanently saves changes.

Q30. db.refresh()?

Answer: Reloads the object from the database.

Q31. db.query()?

Answer: Starts a database query.

Q32. filter()?

Answer: Applies conditions to a query.

Q33. first() vs all()?

Answer: first() returns one record; all() returns a list.

Q34. Without commit()?

Answer: Changes are not permanently saved.

Q35. JWT?

Answer: JSON Web Token used for authentication.

Q36. Why hash passwords?

Answer: To avoid storing plain-text passwords.

Q37. Hashing vs encryption?

Answer: Hashing is one-way; encryption can be reversed with a key.

Q38. bcrypt?

Answer: A secure password hashing algorithm.

Q39. Login verification?

Answer: Hash the entered password and compare it with the stored hash.

Q40. OAuth2PasswordBearer?

Answer: Extracts bearer tokens from requests.

Q41. Protected routes?

Answer: Routes that require a valid authenticated user.

Q42. HTTPException?

Answer: Used to return HTTP error responses.

Q43. Authentication vs authorization?

Answer: Authentication verifies identity; authorization checks permissions.

Q44. CORS?

Answer: Controls which origins may access your API.

Q45. Middleware?

Answer: Code executed before and after requests.

Q46. Input validation?

Answer: Using Pydantic models and type hints.

Q47. Exception handling?

Answer: Using HTTPException or custom handlers.

Q48. Testing APIs?

Answer: Using TestClient, pytest, and Swagger UI.

Q49. Deployment?

Answer: Run with Uvicorn/Gunicorn behind a reverse proxy or cloud platform.

Q50. Professional architecture?

Answer: Separate files for routes, models, schemas, CRUD, database, authentication, and configuration.